



Guia completo da arquitetura

Como o Lavamo funciona, camada por camada — e por que cada decisão foi tomada.

Atividade 3 — Programação Mobile (APP Android + Web Site no mesmo projeto)

Expo SDK 57 · React Native 0.86 · Expo Router · TypeScript

Material de estudo para a apresentação

Sumário

01	Visão geral do projeto	o que é, requisitos e como foram atendidos
02	Tecnologias utilizadas	cada biblioteca e para que serve
03	A arquitetura em camadas	a regra principal do projeto
04	As pastas, uma por uma	app, integration, context, hooks, components, constants, utils, @types
05	Rotas e navegação (Expo Router)	grupos, guards, Android × Web
06	Login, cadastro e cookie	o fluxo completo da autenticação
07	O proxy (auth+api, create+api, logout+api)	o problema, a solução e o teste
08	O "banco de dados" (bd.json + AsyncStorage)	regras de reserva
09	A câmera e o QR Code (recurso nativo)	expo-camera do começo ao fim
10	Android × Web: as 4 técnicas multiplataforma	como a mesma base vira duas interfaces
11	Fluxos completos, passo a passo	do toque na tela até o dado salvo
12	Comparação com o projeto Pokémon	o que foi mantido e o que mudou
13	Decisões tomadas e seus motivos	tabela para revisar rápido
14	Perguntas prováveis do professor	com respostas prontas
15	Glossário	termos técnicos em uma linha

Como usar este guia

Cada capítulo explica o conceito, mostra o trecho real do código com o caminho do arquivo e termina, quando faz sentido, com um quadro  **"Como explicar"** — uma frase curta para você falar na apresentação. O capítulo 14 reúne as perguntas mais prováveis com as respostas.

Visão geral do projeto

O Lavamo gerencia a lavanderia comunitária de um condomínio: cada máquina tem um QR Code, e o morador vê na hora se ela está livre, reserva e depois libera para o próximo.

O problema

Num condomínio com lavanderia compartilhada não existe controle de uso: o morador desce, encontra a máquina ocupada e não sabe de quem é nem se ela vai liberar. O Lavamo resolve isso colando um QR Code em cada máquina. Ao ler o QR, o app mostra o modelo, o ano e o status da máquina.

Situação da máquina	O que o app mostra
Livre	Botão "Utilizar máquina" . Ao tocar, ela fica em uso pelo usuário logado.
Em uso por outra pessoa	"Em uso por <i>fulano</i> desde HH:mm", sem botão.
Em uso por mim	Botão "Cancelar reserva" , que deixa a máquina livre de novo.

Requisitos da atividade × como foram atendidos

O enunciado pede	Como o Lavamo atende	Onde está
APP Android e Web Site no mesmo projeto	Um único projeto Expo gera o app Android e o site. As telas se adaptam por plataforma, e a área logada tem layouts diferentes (abas no Android, barra superior na Web).	<code>src/app/</code> , arquivos <code>*.web.tsx</code>
Integração de login usando cookies	Usa a API do professor. O axios é criado com <code>withCredentials: true</code> , e o JWT fica num cookie <code>HttpOnly</code> que o JavaScript nunca lê.	<code>src/integration/</code> , <code>src/context/</code>
Criação de usuário	Tela de cadastro (usuário, e-mail, CEP, senha e confirmação) chamando <code>POST /create</code> .	<code>src/app/(auth)/register.tsx</code>
Login	Tela de login chamando <code>POST /auth</code> .	<code>src/app/(auth)/index.tsx</code>
Recurso nativo do sistema operacional	Câmera lendo QR Code, com a biblioteca <code>expo-camera</code> e permissão pedida em tempo de execução.	<code>src/components/qr-scanner/</code> , <code>scan.tsx</code>
Qualidade de código e estrutura	Arquitetura em camadas igual à vista em aula (projeto Pokémon), TypeScript <code>strict</code> , lint e typecheck sem erros.	todo o <code>src/</code>

O projeto em números

Telas

Login, Cadastro, Início, Escanear (só Android), Perfil e Detalhe da máquina.

Dados

5 máquinas (LVM-001 a LVM-005) em `bd.json`, com reservas salvas no aparelho.

API externa

API de login do professor: `https://login-p26w.onrender.com/fatec/login/v1`.

Regras

No máximo 1 máquina por usuário, e só quem reservou pode cancelar.

Tecnologias utilizadas

Todas as dependências foram instaladas com `npx expo install`, que escolhe a versão compatível com o SDK do Expo.

Biblioteca	Para que serve no Lavamo
expo (SDK 57)	A plataforma que empacota o React Native e fornece os módulos nativos (câmera, armazenamento etc.). Com ela, o mesmo código roda no Android, no iOS e na Web.
react-native 0.86	Os componentes de interface (<code>View</code> , <code>Text</code> , <code>TextInput</code> ...) que viram componentes nativos no Android.
react-native-web + react-dom	Traduzem esses mesmos componentes para HTML no navegador. É isso que permite o "Web Site no mesmo projeto".
expo-router	Navegação baseada em arquivos: cada arquivo em <code>src/app/</code> é uma tela, e os arquivos <code>_layout.tsx</code> definem os navegadores (Stack, Tabs). Também fornece as API Routes (arquivos <code>+api.ts</code>), usadas no proxy.
axios	Cliente HTTP para falar com a API de login, com interceptors e <code>withCredentials</code> .
@react-native-async-storage/async-storage	Armazenamento chave/valor persistente no aparelho. É o "banco" das reservas. No Android fica na área privada do app; na Web vira o <code>localStorage</code> .
expo-camera	O recurso nativo. Acessa a câmera e lê QR Codes (componente <code>CameraView</code> e hook <code>useCameraPermissions</code>).
@expo/vector-icons + expo-font	Os ícones das abas do Android (Ionicons). O <code>expo-font</code> é exigido pelos ícones.
react-native-safe-area-context	Informa o tamanho da barra de status/entalhe do celular, para o conteúdo não ficar embaixo dela (<code>useSafeAreaInsets</code>).
react-native-screens , expo-linking , expo-constants , expo-status-bar	Dependências exigidas pelo Expo Router: telas nativas, links, constantes do app e cor da barra de status.
TypeScript (strict)	Tipagem estática: o editor e o <code>tsc</code> acusam erros antes de rodar.

Por que Expo e não "React Native puro"?

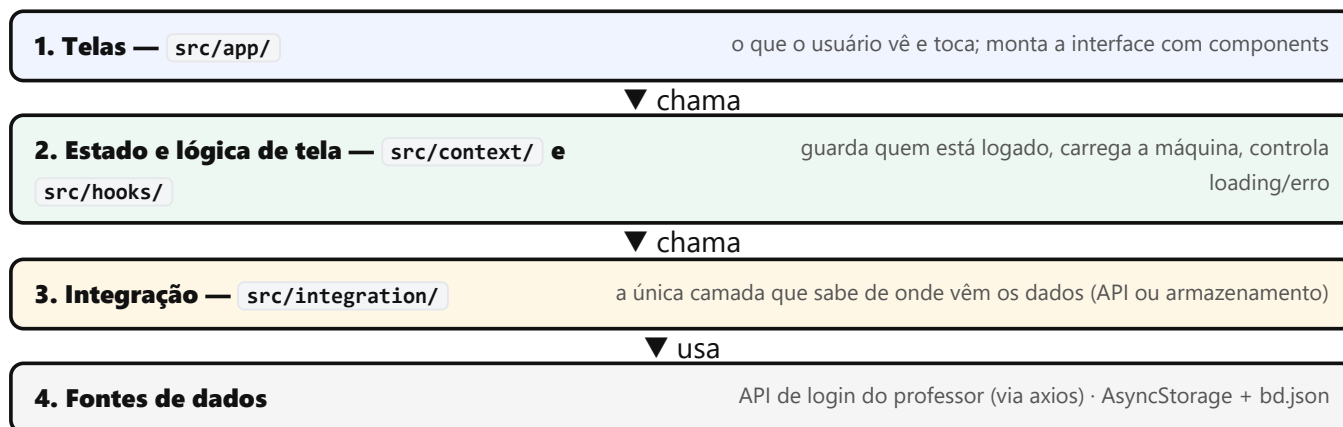
O Expo é hoje o framework recomendado pela própria documentação do React Native. Ele entrega a câmera, o armazenamento e o roteamento prontos e compatíveis entre si, roda no **Expo Go** sem precisar compilar um APK durante o desenvolvimento, e gera a versão Web com o mesmo código.

Arquivos de configuração da raiz

Arquivo	Função
<code>package.json</code>	Lista de dependências e scripts. O <code>"main": "expo-router/entry"</code> diz que o ponto de entrada é o Expo Router, que lê a pasta <code>src/app/</code> .
<code>app.json</code>	Configuração do app: nome, ícones, <code>scheme</code> (<code>lavamo</code>), <code>web.output: "server"</code> (necessário para o proxy) e os plugins , entre eles o do <code>expo-camera</code> com o texto da permissão.
<code>tsconfig.json</code>	Configuração do TypeScript e dos atalhos de import: <code>@/...</code> aponta para <code>src/</code> e <code>@assets/...</code> para <code>assets/</code> .
<code>.env</code> / <code>.env.example</code>	URLs da API. <code>EXPO_PUBLIC_AUTH_API_URL</code> vai para dentro do app (Android); <code>AUTH_API_URL</code> é lida só pelo servidor do proxy. O <code>.env</code> não vai para o git.
<code>bd.json</code>	As 5 máquinas iniciais (a carga do "banco").
<code>eslint.config.js</code>	Regras de lint do Expo (<code>npx expo lint</code>).

A arquitetura em camadas

É a mesma organização do projeto Pokémon visto em aula. A ideia central: **cada camada só conversa com a camada logo abaixo.**



A regra de ouro

Nenhuma tela chama o `axios` ou o `AsyncStorage` diretamente. A tela de login chama `signIn()` do contexto, o contexto chama `login()` da integração, e só a integração faz o `POST`. Da mesma forma, a tela da máquina chama o hook `useMachine`, que chama `reserveMachine()` da integração, que é quem grava no `AsyncStorage`.

Por que separar assim?

- **Troca de fonte de dados sem mexer nas telas.** Se amanhã existir uma API real das máquinas, só o `machineIntegration.ts` muda. As funções continuam com o mesmo nome e retorno, e as telas nem percebem.
- **Responsabilidade única.** Cada arquivo tem um papel claro: a tela desenha, o contexto guarda a sessão, a integração busca dados. Fica mais fácil achar e corrigir um problema.
- **Reuso.** O mesmo `useAuthCookie()` é usado no login, no cadastro, no Início, no Perfil, na barra da Web e nos guards de rota.
- **Testabilidade.** As regras de negócio (limite de 1 reserva, só o dono cancela) estão em funções puras da integração, fáceis de testar isoladamente.

Exemplo real: o caminho de um login



Como explicar

"Usei a arquitetura em camadas vista em aula. As telas ficam em `app` e só desenharam a interface. O estado global, como o usuário logado, fica em `context`. Toda comunicação com dados fica em `integration`, que é a única camada que sabe se o dado vem de uma API ou do armazenamento do aparelho. Assim, se eu trocar a fonte de dados, as telas não mudam."

As pastas, uma por uma

A estrutura completa do `src/`, e depois o papel de cada pasta com os arquivos que ela contém.

```
lavamo/
├─ bd.json # as 5 máquinas iniciais
├─ app.json · package.json · tsconfig.json · .env
├─ assets/images/logo-lavamo.png # logo
├─ docs/qrcodes/LVM-001..005.png # QR Codes para a demonstração
└─ src/
    ├─ app/ # TELAS e ROTAS (Expo Router)
    │   ├── _layout.tsx # raiz: AuthCookieProvider
    │   ├── (auth)/_layout.tsx # guard da área pública
    │   ├── (auth)/index.tsx # Login → URL /
    │   ├── (auth)/register.tsx # Cadastro → URL /register
    │   ├── (app)/_layout.tsx # guard + Stack (Android)
    │   ├── (app)/_layout.web.tsx # guard + barra superior (Web)
    │   ├── (app)/(tabs)/_layout.tsx # abas (Android)
    │   ├── (app)/(tabs)/_layout.web.tsx # sem abas (Web)
    │   ├── (app)/(tabs)/home.tsx # Início → /home
    │   ├── (app)/(tabs)/scan.tsx # câmera → /scan (Android)
    │   ├── (app)/(tabs)/scan.web.tsx # Web: redireciona para /home
    │   ├── (app)/(tabs)/profile.tsx # Perfil → /profile
    │   ├── (app)/machine/[code].tsx # Detalhe → /machine/LVM-001
    │   └─ api/login/v1/ # PROXY (roda no servidor)
    │       ├── auth+api.ts · create+api.ts · logout+api.ts
    │       └─ integration/ # acesso a dados
    │           ├── httpClientCookie.ts · authCookieIntegration.ts · machineIntegration.ts
    │           └─ context/AuthCookieContext.tsx # sessão do usuário
    │   └─ hooks/useMachine.ts # lógica da tela da máquina
    │   └─ components/ # peças de interface reutilizáveis
    │       ├── alert/ (index.ts, .android, .ios, .web, types)
    │       ├── auth-layout/ button/ card/ input/ logo/
    │       └─ machine-code-form/ machine-status/ qr-scanner/ web-navbar/
    └─ constants/colors.ts # paleta de cores
    └─ utils/authProxy.ts # função de repasse do proxy
    └─ @types/machine.ts · user.ts # tipos TypeScript
```

app/ — as telas e as rotas

Para que serve: com o Expo Router, **cada arquivo dentro de `src/app/` vira uma rota** (uma URL na Web, uma tela no Android). Não existe um arquivo central de rotas: a própria estrutura de pastas é a navegação.

Convenção	Significado	Exemplo no Lavamo
<code>_layout.tsx</code>	Não é uma tela: é a "moldura" que envolve as telas daquela pasta e define o navegador (Stack, Tabs ou Slot). É onde colocamos os guards de login.	<code>(app)/_layout.tsx</code>
<code>(nome)</code> — pasta entre parênteses	Grupo: organiza arquivos sem aparecer na URL. <code>(auth)/register.tsx</code> vira <code>/register</code> , não <code>/auth/register</code> .	<code>(auth)</code> , <code>(app)</code> , <code>(tabs)</code>
<code>index.tsx</code>	A rota "raiz" da pasta.	<code>(auth)/index.tsx</code> = <code>/</code>
<code>[code].tsx</code> — colchetes	Rota dinâmica: o trecho da URL vira um parâmetro, lido com <code>useLocalSearchParams()</code> .	<code>/machine/LVM-001</code> → <code>code</code> = <code>"LVM-001"</code>
<code>*.web.tsx</code>	Versão só da Web daquele arquivo. Exige que exista também a versão sem extensão (usada no Android).	<code>_layout.web.tsx</code> , <code>scan.web.tsx</code>
<code>*+api.ts</code>	API Route: não é tela, é código que roda no servidor e responde a pedidos HTTP.	<code>api/login/v1/auth+api.ts</code>

O que fica fora de `app/` (componentes, hooks etc.) não vira rota, e por isso fica em pastas próprias.

integration/ — a camada de dados

Para que serve: é o **único lugar que sabe de onde vêm os dados**. Esconde o axios, as URLs, os códigos HTTP e o AsyncStorage atrás de funções simples, como `login()` ou `reserveMachine()`.

Arquivo	O que faz
<code>httpClientCookie.ts</code>	"Fábrica" de clientes axios configurados para cookie: <code>withCredentials: true</code> , <code>timeout</code> de 60 s e o interceptor que detecta sessão expirada (401/403). Mesmo desenho do Pokémon.
<code>authCookieIntegration.ts</code>	As 3 chamadas da API de login: <code>register</code> (<code>/create</code>), <code>login</code> (<code>/auth</code>) e <code>logout</code> (<code>/logout</code>). Escolhe a URL base por plataforma e valida a resposta.
<code>machineIntegration.ts</code>	O "banco" das máquinas: lê o <code>bd.json</code> , salva no AsyncStorage e aplica as regras (reservar, cancelar, buscar, minha reserva).

context/ — o estado global

Para que serve: guardar informações que **várias telas** precisam ao mesmo tempo, sem ficar passando props de tela em tela. No Lavamo, é a **sessão do usuário**: `AuthCookieContext.tsx` guarda o `user` (userId, username, roles) e expõe `signIn`, `signUp` e `signOut`. Qualquer componente acessa com `useAuthCookie()`.

O `AuthCookieProvider` envolve o app inteiro em `src/app/_layout.tsx`. Por isso ele está disponível em todas as telas.

hooks/ — lógica reutilizável de tela

Para que serve: tirar das telas a lógica de "carregar dado, controlar loading, tratar erro, executar ação". O `useMachine(code, user)` busca a máquina, informa se ela é minha (`isMine`) e oferece `reserve()` e `cancel()` já com as mensagens prontas. A tela `machine/[code].tsx` só desenha o resultado.

Diferença entre context e hook

O **context** guarda um estado **compartilhado pelo app todo** (existe um só usuário logado). O **hook** cria um estado **local** para quem o usa: cada tela de máquina aberta tem o seu próprio `useMachine`.

components/ — peças de interface

Para que serve: blocos visuais reutilizáveis. Seguem o padrão do Pokémon: uma pasta por componente, com `index.tsx` (o componente) e, quando precisa, `styles.ts` (os estilos). Os componentes base **estendem as props nativas** (`TouchableOpacityProps`, `TextInputProps`) e repassam o resto com `...rest`.

Componente	Função
<code>alert/</code>	Alerta com uma implementação por plataforma : Android usa o alerta nativo do sistema; Web usa um Modal próprio. Veja o capítulo 10.
<code>button/</code>	Botão com as variantes <code>primary</code> (preto) e <code>outline</code> (borda), além de <code>loading</code> .
<code>input/</code>	Campo de texto com <code>label</code> opcional.
<code>card/</code>	Caixa branca com borda. É a base de quase todas as telas.
<code>logo/</code>	A logo com a proporção correta (a largura é calculada pela altura).
<code>auth-layout/</code>	A moldura de login e cadastro. Na Web larga, mostra o painel preto lateral.
<code>machine-code-form/</code>	Campo "código da máquina" + botão Buscar, com validação do formato <code>LVM-000</code> .
<code>machine-status/</code>	Selo "LIVRE" / "EM USO".
<code>qr-scanner/</code>	A câmera lendo QR Code (o recurso nativo). Veja o capítulo 9.
<code>web-navbar/</code>	A barra superior da versão Web (Início, Perfil, usuário, Sair).

constants/ — valores fixos

Para que serve: centralizar valores usados em muitos lugares. O `colors.ts` define toda a paleta (preto, branco, cinzas, borda, overlay). Nenhuma tela escreve uma cor "solta" como `'#000'`: todas usam `Colors.black`. Para mudar o tema, basta mudar um arquivo.

utils/ — funções utilitárias

Para que serve: funções de apoio que não são tela, nem estado, nem integração com o app. O `authProxy.ts` contém a função `forwardToAuthApi()`, usada pelos três arquivos do proxy. Ela fica em `utils/` (e não em `app/`) porque tudo o que está em `app/` vira rota, e ela é só uma função auxiliar compartilhada.

@types/ — os tipos de domínio

Para que serve: descrever o formato dos dados do negócio, para o TypeScript conferir o código.

src/@types/machine.ts

```
export interface Reservation { userId: string; username: string; reservedAt: string; }

export interface Machine {
  code: string;           // "LVM-001" - é o que vai no QR Code
  model: string;
  year: number;
  available: boolean;     // true = livre
  reservedBy: Reservation | null; // quem está usando
}
```

O `user.ts` define `User { userId, username, roles }`, exatamente o formato que a API de login devolve.

Rotas e navegação (Expo Router)

Como os arquivos viram telas, como o app impede o acesso sem login e como a navegação muda entre Android e Web.

Mapa de rotas

URL / tela	Arquivo	Quem acessa
/ — Login	(auth)/index.tsx	só quem não está logado
/register — Cadastro	(auth)/register.tsx	só quem não está logado
/home — Início	(app)/(tabs)/home.tsx	logado
/scan — Escanear	(app)/(tabs)/scan.tsx · Web: scan.web.tsx	logado (a Web redireciona para /home)
/profile — Perfil	(app)/(tabs)/profile.tsx	logado
/machine/LVM-001 — Detalhe	(app)/machine/[code].tsx	logado
POST /api/login/v1/...	api/login/v1/*+api.ts	não é tela: é o proxy (só a Web usa)

Os guards (proteção de rotas)

Os `_layout.tsx` dos grupos funcionam como "porteiros". Eles leem `isAuthenticated` do contexto e, se a pessoa não pode estar ali, retornam um `<Redirect>`.

src/app/(app)/_layout.tsx — área logada

```
export default function AppLayout() {
  const { isAuthenticated } = useAuthCookie();

  if (!isAuthenticated) {
    return <Redirect href="/" />; // sem login → volta para o Login
  }
  return <Stack> ... </Stack>;
}
```

src/app/(auth)/_layout.tsx — área pública

```
if (isAuthenticated) {
  return <Redirect href="/home" />; // já logado → vai para o Início
}
return <Stack screenOptions={{ headerShown: false }} />;
```

Consequência importante: as telas não navegam depois do login

A tela de login só chama `signIn()`. Quando dá certo, o contexto preenche o `user`, o `isAuthenticated` vira `true`, o React redesenha o `(auth)/_layout`, e ele mesmo redireciona para `/home`. O logout funciona igual ao contrário: `signOut()` limpa o usuário, e o `(app)/_layout` manda para `/`. Há **um único lugar** decidindo o acesso, e não existe navegação duplicada.

A navegação no Android × na Web

ANDROID Abas embaixo + Stack

`(app)/_layout.tsx` cria um **Stack** (telas empilhadas com botão voltar). Dentro dele, `(tabs)/_layout.tsx` cria as **Tabs**: Início, Escanear e Perfil, com ícones Ionicons. A tela da máquina abre **por cima** das abas, com cabeçalho "Máquina" e seta de voltar.

O `Tabs` é importado de `expo-router/js-tabs`: no SDK 57, importar de `expo-router` está marcado como descontinuado.

WEB Barra superior + conteúdo centralizado

`(app)/_layout.web.tsx` desenha a **WebNavbar** (logo, Início, Perfil, usuário, Sair) e um conteúdo com largura máxima de 1040 px. `(tabs)/_layout.web.tsx` é só um `<Slot />`, porque na Web não há abas. A tela da máquina tem o link "← Voltar ao início".

Stack, Tabs e Slot: `Stack` empilha telas (dá para voltar); `Tabs` mostra uma barra de abas; `Slot` apenas renderiza a rota atual, sem navegador visual. Na Web, o "voltar" é o do próprio navegador.

Login, cadastro e cookie

Como a autenticação funciona de ponta a ponta, desde o que é um cookie até cada linha do código envolvido.

O que é um cookie de sessão

Cookie é um pequeno dado que o **servidor manda** na resposta (cabeçalho `Set-Cookie`) e que o **cliente guarda e devolve automaticamente** nos próximos pedidos àquele servidor (cabeçalho `Cookie`). A API do professor usa isso para a sessão: no login, ela devolve um cookie com um token JWT dentro.

Resposta real da API (testada durante o desenvolvimento):

```
HTTP/1.1 200 OK
Set-Cookie: access_token=eyJhbGciOiJIUzI1NiJ9...; Path=/; Max-Age=120; HttpOnly; SameSite=Lax

{"userId":"65b08654-bba9-4752-b198-0079f7ddf527","username":"lavamo_teste","roles":["USER"]}
```

Atributo	O que significa	Consequência no Lavamo
<code>access_token=...</code>	O nome e o valor do cookie: um JWT com o usuário e a validade.	É a "prova" de que a pessoa está logada.
<code>HttpOnly</code>	O JavaScript não consegue ler esse cookie (<code>document.cookie</code> não o mostra). Só o navegador/app o envia.	Mais seguro contra roubo de sessão (XSS). Por outro lado, o app não consegue saber sozinho se ainda está logado.
<code>SameSite=Lax</code>	O navegador só guarda/envia o cookie em pedidos feitos a partir do mesmo site da API.	É o motivo do proxy (capítulo 7).
<code>Max-Age=120</code>	O cookie vale por 120 segundos.	A sessão da API expira em 2 minutos.
<code>Path=/</code>	Vale para todos os caminhos do servidor.	—

Camada 1 — o cliente HTTP com cookie

src/integration/httpClientCookie.ts

```
const PUBLIC_ROUTES = ['/auth', '/create'];

export function createApiClient(baseUrl: string) {
  // withCredentials: o navegador/app guarda e envia o cookie da sessão sozinho.
  // timeout alto porque o servidor gratuito do Render demora para "acordar".
  const instance = axios.create({ baseUrl, withCredentials: true, timeout: 60000 });

  instance.interceptors.response.use(
    (response) => response,
    (error) => {
      const status = error?.response?.status;
      const isPublicRoute = PUBLIC_ROUTES.includes(error?.config?.url);
      // A API do professor responde 403 (e não 401) quando o cookie é inválido/expirou.
      if ((status === 401 || status === 403) && !isPublicRoute) {
        onUnauthorized?.(); // avisa o contexto: sessão acabou
      }
      return Promise.reject(error);
    }
  );
  return instance;
}
```

- `withCredentials: true` : sem isso, o navegador **não guarda nem envia** cookies em pedidos para outro endereço. É o que torna o login "via cookie".
- `timeout: 60000` : o Render (hospedagem gratuita da API) "dorme" sem uso, e o primeiro pedido pode levar ~50 s. Com o padrão, o pedido falharia antes.
- **Interceptor**: é uma função que roda em **toda resposta com erro**, antes de ela chegar a quem chamou. Se for 401/403 numa rota protegida, chama o `onUnauthorized`, que o contexto registrou (limpa a sessão e volta ao login).
- **Por que 403 e não só 401 (diferença para o Pokémon)**: a API do professor responde **403** quando o cookie não é válido.
- **Por que ignorar `/auth` e `/create`** : senha errada e usuário duplicado **também dão 403**. Sem essa exceção, errar a senha seria tratado como "sessão expirada".
- `setUnauthorizedHandlerCookie(handler)` : a integração não conhece o `router` nem o estado do React. Por isso o contexto "injeta" o que fazer quando a sessão cai. Assim a integração continua independente da interface.

Camada 2 — as chamadas da API de login

src/integration/authCookieIntegration.ts

```
const AUTH_BASE_URL = Platform.OS === 'web'
  ? '/api/login/v1' // Web → proxy (mesmo endereço do site)
  : process.env.EXPO_PUBLIC_AUTH_API_URL || DEFAULT_AUTH_API_URL; // Android → API direto

const authApi = createApiCookie(AUTH_BASE_URL);

export const register = async (data: RegisterRequest) => { // { username, password, email, cep }
  const response = await authApi.post('/create', data);
  return parseSession(response.data);
};

export const login = async (data: AuthRequest) => { ... authApi.post('/auth', data) ... };
export const logout = async () => { await authApi.post('/logout'); };
```

- **Uma decisão de plataforma num único lugar**: o código das três funções é igual para Android e Web; só muda a URL base. Na Web, `'/api/login/v1'` é um endereço **relativo**, ou seja, o próprio servidor do site (`localhost:8081`).
- **Fallback de URL**: se não houver `.env`, o app usa a URL da API online. O projeto funciona "clonou, rodou".
- `isAuthCookieResponse()` / `parseSession()` : conferem, **em tempo de execução**, se a resposta realmente tem `userId`, `username` e `roles` antes de confiar nela. O tipo TypeScript só vale na compilação; essa checagem protege contra uma API que devolva algo inesperado.
- **Os tipos** `RegisterRequest` e `AuthRequest` seguem exatamente o `curl` do enunciado (o cadastro inclui o `cep`).

Camada 3 — o contexto da sessão

src/context/AuthCookieContext.tsx (resumo)

```
const [user, setUser] = useState<User | null>(null); // sessão em memória

useEffect(() => {
  setUnauthorizedHandlerCookie(() => { clearSession(); router.replace("/"); });
}, []);

async function signIn(username, password) {
  try {
    const response = await loginApi({ username, password });
    setUser(response); // isAuthenticated vira true → guard redireciona
    return { ok: true };
  } catch (err) {
    if (isNetworkError(err)) return { ok: false, error: 'Servidor indisponível...' };
    return { ok: false, error: 'Usuário ou senha incorretos.' };
  }
}

// signUp: igual, chamando registerApi — o /create já devolve o usuário E o cookie,
// então a pessoa entra direto no app depois do cadastro.
async function signOut() {
  try { await logoutApi(); } catch {} finally { clearSession(); }
}

// value: { isAuthenticated: !!user, user, isLoading, signIn, signUp, signOut }
```

- **O contexto nunca guarda o token.** Ele guarda apenas os dados do usuário (`userId`, `username`, `roles`) que vêm no corpo da resposta. O token está no cookie, que o JS não pode ler.
- `isAuthenticated` é derivado (`!!user`): não existe um segundo estado que possa ficar dessincronizado.
- **Mensagens de erro próprias:** a API responde 403 com o corpo **vazio**, então o app cria as mensagens. `isNetworkError` distingue "o servidor não respondeu" de "o servidor recusou".
- `signOut` com `finally`: mesmo que o cookie já tenha expirado e o `/logout` falhe, a sessão local é limpa.

Por que recarregar a página volta para o login?

A sessão do app fica **em memória** (no `useState`). Recarregar a página ou fechar o app apaga a memória. Para "lembrar" o login, o app precisaria perguntar à API "quem sou eu?", mas **essa rota não existe**, e o cookie `HttpOnly` não pode ser lido pelo JS. Guardar o usuário no `AsyncStorage` faria o app parecer logado com um cookie que já expirou (dura só 2 minutos). Por isso a decisão foi manter o mesmo comportamento do Pokémon: sessão em memória.

Camada 4 — as telas

Login (`(auth)/index.tsx`): valida se os campos estão preenchidos, chama `signIn` e mostra o `Alert` em caso de erro. Enquanto espera, o botão mostra um carregando e o texto "Conectando ao servidor... o primeiro acesso pode demorar".

Cadastro (`(auth)/register.tsx`): validações no front antes de chamar a API.

Validação	Regra
Campos obrigatórios	usuário, e-mail, CEP, senha e confirmação
E-mail	expressão regular <code>/^[^\s@]+@[^\s@]+\.[^\s@]+\$/</code>
CEP	máscara <code>00000-000</code> na digitação; exige 8 dígitos; é enviado só com números (<code>onlyDigits</code>)
Senha	mínimo de 4 caracteres, e igual à confirmação

Por que o cookie funciona no Android sem proxy?

A regra `SameSite` é uma regra **dos navegadores**. No Android, o app não é um navegador: os pedidos HTTP saem pelo módulo de rede nativo do React Native, que tem um "pote de cookies" (*cookie jar*). Ele guarda o `Set-Cookie` da resposta e o envia nos pedidos seguintes para o mesmo servidor, sem a restrição de "mesmo site". Por isso o Android chama `https://login-p26w.onrender.com` direto.

Como explicar

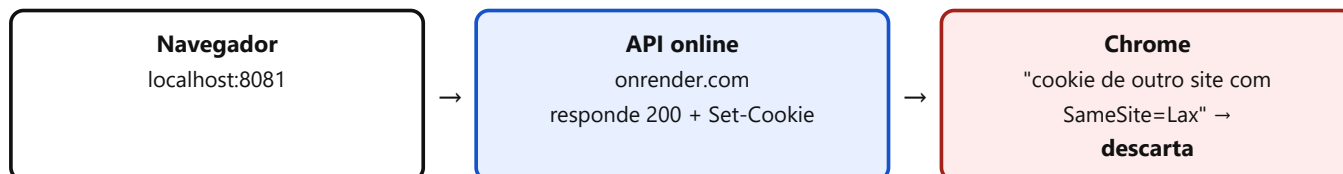
"O login é por cookie: a API devolve um cookie `HttpOnly` com o JWT, e o axios com `withCredentials: true` faz o navegador ou o app guardá-lo e reenviá-lo sozinho. O JavaScript nunca vê o token. No contexto eu guardo só os dados do usuário que vêm no corpo da resposta. Um interceptor detecta 401/403 nas rotas protegidas e desloga automaticamente."

O proxy — auth+api, create+api e logout+api

A decisão técnica mais importante do projeto. Aqui estão o problema, a prova de que ele existia, a solução e cada linha do código.

O problema: o navegador jogava o cookie fora

Na Web, o site roda em `http://localhost:8081` e a API em `https://login-p26w.onrender.com`. Para o navegador, são **dois sites diferentes**. Como o cookie vem com `SameSite=Lax`, o navegador **não guarda** um cookie recebido de outro site num pedido feito por JavaScript (`fetch` / `axios`), e também não o envia.



A prova: o teste feito no Chrome

Durante o desenvolvimento foi feito um teste real no Chrome: uma página em `localhost:8081` fez o login na API online e, em seguida, chamou `/logout`, que só responde 200 se receber o cookie.

Cenário	/auth	/logout	Conclusão
Navegador → API online direto	200	403	O cookie não foi guardado
Terminal (<code>curl</code>) → API online	200	200	A API funciona; quem bloqueia é o navegador
Navegador → proxy → API online	200	200	Com o proxy, o cookie funciona

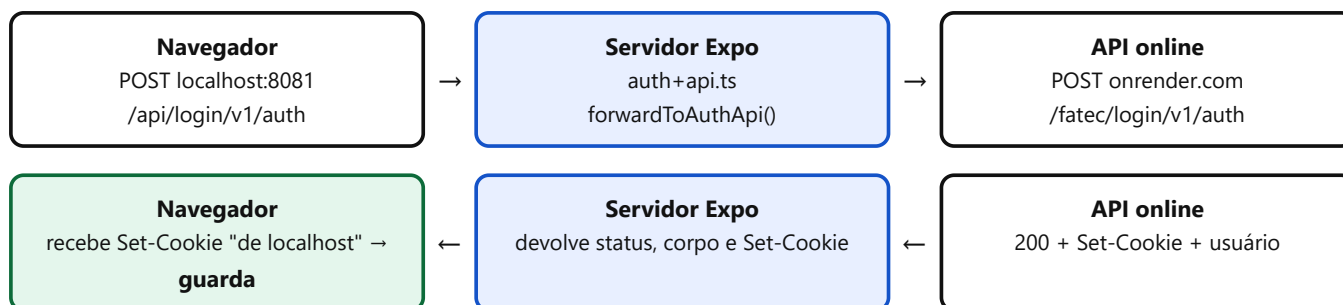
SameSite não é CORS (pergunta clássica)

CORS decide se o JavaScript de um site pode **ler a resposta** de outro site. A API do professor **libera** o CORS para `localhost:8081` (`access-control-allow-origin` + `allow-credentials`), por isso o login "parecia funcionar" e o app recebia o usuário.

SameSite é outra regra, do cookie: decide se o navegador **guarda e envia o cookie** em pedidos entre sites diferentes. Mesmo com o CORS liberado, o `SameSite=Lax` bloqueia o cookie. É por isso que o problema só aparecia no `/logout`.

A solução: um intermediário no mesmo endereço do site

O **proxy** é um intermediário. Em vez de o navegador falar com o `onrender.com`, ele fala com o **próprio servidor do site** (`localhost:8081/api/login/v1/...`), e esse servidor repassa o pedido para a API.



1

O app na Web faz `POST /api/login/v1/auth` para o próprio endereço do site.

- 2 O Expo Router encontra o arquivo `src/app/api/login/v1/auth+api.ts` e executa a função `POST` **no servidor** (Node.js, no seu PC), não no navegador.
- 3 Ela chama `forwardToAuthApi(request, '/auth')`, que faz o pedido para a API online. Entre servidor e API **não existe SameSite nem CORS**, porque essas regras só existem dentro de navegadores.
- 4 A API responde com o usuário e o `Set-Cookie`.
- 5 O proxy devolve ao navegador o mesmo status, o mesmo corpo e o mesmo `Set-Cookie`.
- 6 Como o cookie **não tem o atributo Domain**, o navegador o associa a quem respondeu: `localhost`, que é o mesmo site. O cookie é aceito.
- 7 No `/logout`, o navegador envia o cookie para `localhost`, o proxy o copia para a API, e a API o reconhece.

Os três arquivos de rota

Cada arquivo é uma **API Route** do Expo Router. O nome do arquivo define a URL, e a função exportada define o método HTTP aceito. Os três são idênticos, mudando só o caminho repassado.

`src/app/api/login/v1/auth+api.ts`

```
import { forwardToAuthApi } from '@utils/authProxy';

// Proxy (Web): POST /api/Login/v1/auth → API de Login /fatec/login/v1/auth
export function POST(request: Request) {
  return forwardToAuthApi(request, '/auth');
}
```

Arquivo	URL que o navegador chama	Para onde é repassado	Usado por
<code>auth+api.ts</code>	<code>POST /api/login/v1/auth</code>	<code>.../fatec/login/v1/auth</code>	<code>login()</code> → tela de Login
<code>create+api.ts</code>	<code>POST /api/login/v1/create</code>	<code>.../fatec/login/v1/create</code>	<code>register()</code> → tela de Cadastro
<code>logout+api.ts</code>	<code>POST /api/login/v1/logout</code>	<code>.../fatec/login/v1/logout</code>	<code>logout()</code> → botão Sair

- **+api no nome** diz ao Expo Router que o arquivo é uma rota de servidor, e não uma tela.
- `export function POST`: só responde a POST. Um GET nesse endereço não é atendido.
- `request: Request` é o objeto padrão da Web (Fetch API), com cabeçalhos e corpo do pedido do navegador.
- **Os caminhos espelham os da API** (`/auth`, `/create`, `/logout`). Por isso o `authCookieIntegration.ts` usa exatamente o mesmo código nas duas plataformas.
- **Não é um proxy aberto**: só existem esses 3 caminhos fixos, para uma única API. Ninguém consegue usar o servidor para acessar outros sites.

A função de repasse, linha a linha

src/utils/authProxy.ts

```
const AUTH_API_URL = process.env.AUTH_API_URL || 'https://login-p26w.onrender.com/fatec/login/v1';

export async function forwardToAuthApi(request: Request, path: string) {
  // (1) copia só os cabeçalhos necessários do navegador
  const headers = new Headers();
  const contentType = request.headers.get('content-type');
  const cookie = request.headers.get('cookie'); // o cookie da sessão (logout)
  if (contentType) headers.set('content-type', contentType);
  if (cookie) headers.set('cookie', cookie);

  const body = await request.text(); // (2) o JSON {username, password...}

  try {
    // (3) servidor → API: aqui não existe SameSite nem CORS
    const upstream = await fetch(`${AUTH_API_URL}${path}`, { method: 'POST', headers, body: body || undefined });

    // (4) monta a resposta para o navegador
    const responseHeaders = new Headers();
    const upstreamType = upstream.headers.get('content-type');
    if (upstreamType) responseHeaders.set('content-type', upstreamType);
    for (const setCookie of upstream.headers.getSetCookie()) {
      responseHeaders.append('set-cookie', setCookie); // (5) repassa o cookie
    }
    return new Response(await upstream.arrayBuffer(), { status: upstream.status, headers: responseHeaders });
  } catch {
    return Response.json({ message: 'API de login indisponível.' }, { status: 502 }); // (6)
  }
}
```

1. **Cabeçalhos:** copia só o `content-type` (para a API saber que é JSON) e o `cookie` (necessário no logout). Não repassa todos os cabeçalhos do navegador, para não mandar informação desnecessária.
2. **Corpo:** lê o texto do pedido (o JSON com usuário e senha) e o repassa sem alterações.
3. **Pedido para a API:** um `fetch` feito pelo servidor. Usa `AUTH_API_URL` do `.env` (sem o prefixo `EXPO_PUBLIC_`, porque só o servidor precisa dela), com a API online como padrão.
4. **Resposta:** mantém o mesmo **status** (200, 201, 403...), então o app no navegador trata erros igual ao Android.
5. `getSetCookie()`: devolve todos os `Set-Cookie` da API, e cada um é repassado. **É esta linha que faz o cookie chegar ao navegador.**
6. **Erro de rede:** se a API estiver fora do ar, o proxy responde **502 (Bad Gateway)** em vez de quebrar.

O que faz o proxy existir: `web.output: "server"`

No `app.json`, `"web": { "output": "server" }` diz ao Expo que a versão Web tem uma parte que roda em servidor. É isso que ativa as API Routes. No desenvolvimento, quem roda o proxy é o próprio `npx expo start`. Não há nada extra para instalar.

E o Android? Não usa o proxy.

No Android, `AUTH_BASE_URL` é a URL completa da API online, e os pedidos vão direto. O proxy existe **só por causa de uma regra do navegador**. Onde essa regra não existe, ele seria só um passo a mais.

Por que o proxy, e não só a API online direta?

Opção	Cookie Android	Cookie Web	Contras
API online direta	✓	✗ (testado)	A atividade pede "login usando cookies"; na Web o cookie seria descartado, e o professor poderia verificar isso no DevTools.
Docker local (imagem do professor)	✓	✓	Exige Docker rodando, PC ligado, celular na mesma rede, IP no <code>.env</code> , firewall liberado e HTTP (não HTTPS) no Android.
API online + proxy ✓ escolhida	✓	✓ (testado)	Poucos arquivos a mais. Continua usando a API online oferecida pelo professor, sem instalar nada.

🔧 Como explicar

"A API manda o cookie com `SameSite=Lax`. Como na Web o site está em `localhost` e a API em `onrender.com`, o navegador descarta o cookie; eu testei, e o logout dava 403. Então criei um proxy com as API Routes do Expo Router: o navegador chama `/api/login/v1/auth` no próprio site, o servidor repassa para a API, e devolve a resposta com o `Set-Cookie`. Para o navegador, o cookie passa a vir do mesmo endereço e é aceito. No Android não existe essa regra, então ele chama a API direto."

O "banco de dados": bd.json + AsyncStorage

Os dados das máquinas são mockados (simulados), mas com regras de negócio reais e com persistência.

As duas peças

bd.json — a carga inicial

Arquivo na raiz com as 5 máquinas. É importado no código (`import seed from '../..bd.json'`), e o Metro (o empacotador do Expo) o coloca **dentro do app**. Por isso ele é **somente leitura**: o app não consegue alterar esse arquivo.

AsyncStorage — onde as mudanças ficam

Armazenamento chave/valor persistente no próprio aparelho. **Android**: na área privada do app. **Web**: no `localStorage` do navegador. Os dados continuam lá depois de fechar o app.

Por que não gravar direto no bd.json?

O `bd.json` fica no seu PC. O app Android roda **no celular**, que não tem acesso à pasta do PC, e o arquivo vai empacotado dentro do app, só para leitura. Na Web, o navegador, por segurança, não pode escrever arquivos no computador. Gravar num arquivo exigiria um servidor rodando no PC (como um json-server). A decisão foi manter o projeto simples: o `bd.json` é a carga inicial e o AsyncStorage guarda as alterações.

O formato dos dados

bd.json (trecho)

```
{ "machines": [
  { "code": "LVM-001", "model": "Brastemp BWK11", "year": 2021, "available": true, "reservedBy": null },
  ... LVM-002 (Electrolux), LVM-003 (Consul), LVM-004 (Samsung), LVM-005 (LG)
] }
```

Depois de uma reserva, a máquina fica assim no AsyncStorage (chave `@Lavamo:machines`, que guarda as 5 máquinas como texto JSON):

```
{ "code": "LVM-001", ..., "available": false,
  "reservedBy": { "userId": "65b08654-...", "username": "rafael", "reservedAt": "2026-09-27T14:29:00.000Z" } }
```

O `userId` e o `username` vêm **da resposta do login da API**, e é por isso que o app sabe quem é o dono da reserva.

Como a carga funciona

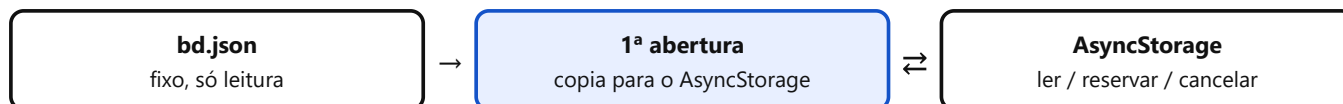
src/integration/machineIntegration.ts

```
const STORAGE_KEY = '@Lavamo:machines';

async function loadMachines() {
  const stored = await AsyncStorage.getItem(STORAGE_KEY);
  if (stored) return JSON.parse(stored); // já existe → usa o que está salvo

  const machines = seedMachines(); // 1ª vez → copia do bd.json
  await saveMachines(machines);
  return machines;
}

function seedMachines() { return seed.machines.map((m) => ({ ...m })); } // cópia, não altera o import
```



As funções e as regras de negócio

Função	O que faz / regra	Quem usa
<code>normalizeCode(code)</code>	Remove espaços e passa para maiúsculas: " lvm-001 " → "LVM-001".	busca, QR, validações
<code>MACHINE_CODE_PATTERN</code>	Regex <code>/^LVM-\d{3}\$/</code> : "LVM-" + exatamente 3 dígitos.	formulário e câmera
<code>getMachineByCode(code)</code>	Retorna a máquina. Se não existir, lança <code>MachineError("Máquina X não encontrada.")</code> .	<code>useMachine</code>
<code>getMyReservation(userId)</code>	Retorna a máquina reservada pelo usuário, ou <code>null</code> .	card "Minha reserva" do Início
<code>reserveMachine(code, user)</code>	① a máquina existe; ② está livre (senão: "já está em uso por..."); ③ o usuário não tem outra reserva (limite de 1). Grava <code>available: false</code> e <code>reservedBy</code> .	botão "Utilizar máquina"
<code>cancelReservation(code, userId)</code>	Só cancela se <code>reservedBy.userId === userId</code> (só o dono). Volta para <code>available: true</code> e <code>reservedBy: null</code> .	botão "Cancelar reserva"

- `MachineError` é uma classe de erro própria. Com ela o hook distingue um erro **esperado** de regra de negócio (mostra a mensagem) de um erro **inesperado** (mostra uma mensagem genérica).
- **As regras ficam na integração, e não na tela.** Mesmo que alguém chamasse a função de outro lugar, a regra seria respeitada.

O hook que as telas usam

src/hooks/useMachine.ts (resumo)

```

export function useMachine(code, user) {
  // estados: machine, isLoading, isSaving, error
  useEffect(() => {
    // carrega ao abrir a tela
    let active = true;
    // evita atualizar estado de tela já fechada
    getMachineByCode(code).then(...).catch(...).finally(...);
    return () => { active = false; };
  }, [code]);

  const reserve = () => runAction(() => reserveMachine(code, user), 'Máquina reservada!...');
  const cancel = () => runAction(() => cancelReservation(code, user.userId), 'Reserva cancelada...');
  const isMine = machine?.reservedBy?.userId === user?.userId;
  return { machine, isLoading, isSaving, isMine, error, reserve, cancel, reload };
}
  
```

O `runAction` liga o `isSaving` (o botão mostra carregando), executa a ação e devolve `{ ok, message }` para a tela mostrar no `Alert`. Se der erro, ele **recarrega** a máquina, para a tela mostrar o estado real.

A tela `machine/[code].tsx` escolhe o que mostrar com três condições: `machine.available` → "Utilizar"; `isMine` → "Cancelar reserva"; senão → "Em uso por fulano".

A limitação (e como falar dela)

Situação	Funciona?
Mesmo aparelho: A reserva, sai, B entra e vê "em uso por A"	✓
A reserva no celular e B olha no navegador (ou em outro celular)	✗ cada aparelho tem a própria cópia dos dados

Como explicar

"Os dados das máquinas são mockados. O `bd.json` é a carga inicial e as reservas ficam no `AsyncStorage` do aparelho, com as regras de negócio no `machineIntegration.ts`. Como isso fica isolado na camada de integração, trocar por uma API real das máquinas seria mudar só esse arquivo: as funções fariam `axios.get/patch`, e as telas continuariam iguais."

A câmera e o QR Code (recurso nativo)

O recurso nativo exigido pela atividade. É usado só no Android, com a biblioteca **expo-camera**.

A biblioteca: expo-camera

- **Pacote:** `expo-camera` (versão ~57.0.5, compatível com o SDK 57), instalado com `npx expo install expo-camera`.
- **O que oferece:** o componente `<CameraView>` (mostra a imagem da câmera e lê códigos), o hook `useCameraPermissions()` (consulta e pede a permissão) e o tipo `BarcodeScanningResult` (o resultado da leitura).
- **Por que ela:** é o módulo **oficial** do Expo para câmera. Já vem no Expo Go (não precisa compilar um APK para testar), e a leitura de QR é feita pelo próprio módulo, sem biblioteca extra.
- **Por que conta como recurso nativo:** a câmera é hardware do aparelho, controlado pelo sistema operacional. O app precisa da **permissão do Android** para usá-la, e a leitura do código é feita em código nativo, fora do JavaScript.

Onde ela aparece no projeto

Arquivo	Papel
<code>app.json</code> → <code>plugins</code>	Adiciona a permissão nativa de câmera e ativa o leitor de códigos.
<code>src/app/(app)/(tabs)/scan.tsx</code>	A tela: pede a permissão, mostra a câmera e decide o que fazer com o código lido.
<code>src/components/qr-scanner/index.tsx</code>	O componente que usa o <code>CameraView</code> , com moldura e trava contra leituras repetidas.
<code>src/app/(app)/(tabs)/scan.web.tsx</code>	Na Web, a rota <code>/scan</code> só redireciona para <code>/home</code> .
<code>src/app/(app)/(tabs)/home.tsx</code>	O botão "Ler QR Code" aparece só fora da Web (<code>!isWeb</code>).
<code>docs/qrcodes/LVM-00X.png</code>	Os QR Codes das 5 máquinas, para a demonstração.

1. A configuração nativa no app.json

`app.json`

```
[ "expo-camera", {  
  "cameraPermission": "O Lavamo usa a câmera para ler o QR Code da máquina de lavar.",  
  "barcodeScannerEnabled": true  
}]
```

Um **config plugin** altera a configuração nativa quando o app é gerado. No Android, ele adiciona a permissão `CAMERA` ao manifesto (`AndroidManifest.xml`). O texto do `cameraPermission` é a explicação exibida no pedido do iOS; no Android, a janela usa o texto padrão do sistema, e por isso a tela Escanear mostra a própria explicação antes de pedir. O `barcodeScannerEnabled` ativa o leitor de códigos. As pastas `android/` e `ios/` **não são editadas à mão**; o Expo as gera a partir do `app.json` (Continuous Native Generation).

2. A permissão em tempo de execução

src/app/(app)/(tabs)/scan.tsx

```
const [permission, requestPermission] = useCameraPermissions();

if (!permission) return <View />; // ainda consultando o sistema

if (!permission.granted) { // sem permissão → explica e oferece o botão
  return ( ... <Button title="Permitir câmera" onPress={requestPermission} />
    {!permission.canAskAgain && <Text>Libere nas configurações do Android...</Text>} );
}

return <QrScanner active={isFocused} onScan={handleScan} />; // com permissão → câmera
```

- `permission` começa `null` enquanto o app consulta o sistema; depois diz se foi concedida (`granted`).
- `requestPermission` abre a janela nativa do Android: "Permitir que o Lavamo tire fotos e grave vídeos?".
- `canAskAgain` : se o usuário negou e marcou "não perguntar de novo", o Android não mostra mais a janela. O app então explica como liberar nas configurações e lembra que dá para digitar o código.
- O app **não pede a câmera logo ao abrir**, só quando o usuário entra na aba Escanear, e explica o motivo antes. Isso é uma boa prática de permissões.

3. A leitura do QR Code

src/components/qr-scanner/index.tsx

```
const locked = useRef(false); // trava contra leituras repetidas

function handleScanned({ data }: BarcodeScanningResult) {
  if (locked.current) return;
  locked.current = true;
  onScan(data); // data = o texto dentro do QR, ex.: "LVM-002"
  setTimeout(() => { locked.current = false; }, 2000);
}

{active && (
  <CameraView
    style={StyleSheet.absoluteFill}
    facing="back" // câmera traseira
    barcodeScannerSettings={{ barcodeTypes: ['qr'] }} // só QR Code
    onBarcodeScanned={handleScanned} // chamado a cada código detectado
  />
)}
```

- `barcodeTypes: ['qr']` : ignora códigos de barras de produtos e lê só QR.
- **A trava (`useRef`)**: a câmera detecta o mesmo QR **várias vezes por segundo**. Sem a trava, o app abriria a tela da máquina várias vezes. O `useRef` guarda um valor que não redesenha a tela ao mudar. Depois de 2 s, a leitura é liberada.
- `active={isFocused}` : `useIsFocused()` diz se a aba está visível. A câmera **só fica ligada com a aba Escanear aberta**, o que economiza bateria, respeita a privacidade e evita leituras enquanto a pessoa está em outra tela.
- **A moldura** é uma `View` desenhada por cima com `pointerEvents="none"`, para não bloquear toques.

4. O que o QR Code precisa conter

O QR carrega **só o código da máquina, em texto puro**: `LVM-002`. Depois da leitura:

src/app/(app)/(tabs)/scan.tsx

```
function handleScan(data: string) {
  const code = normalizeCode(data); // " lvm-002 " → "LVM-002"
  if (!MACHINE_CODE_PATTERN.test(code)) { // não é LVM-000 → alerta
    setIsAlertVisible(true); // "QR Code não é de uma máquina Lavamo"
    return;
  }
  router.push(`/machine/${code}`); // abre o detalhe da máquina
}
```

Conteúdo do QR	Resultado
LVM-002 , lvm-002 , LVM-002	✅ abre a máquina (o código é normalizado)
LVM-999	⚠️ formato válido, mas a tela mostra "Máquina LVM-999 não encontrada."
https://... , LVM-2 , texto qualquer	❌ alerta "QR Code inválido"

Digitar o código (`MachineCodeForm`) usa a **mesma** normalização e a **mesma** regex. QR e digitação levam à mesma rota `/machine/LVM-00X` , e daí em diante o fluxo é idêntico.

5. Por que a Web não tem QR Code?

Foi uma **decisão de produto**: no site o usuário só digita o código (o `expo-camera` até funcionaria no navegador). A regra foi aplicada em três pontos:

- `scan.web.tsx` : se alguém abrir `/scan` no navegador, é redirecionado para `/home` .
- `home.tsx` : o botão "Ler QR Code" e o texto "ou" aparecem só com `!isWeb` .
- `web-navbar` : a barra da Web tem só Início e Perfil, sem Escanear.

🔑 Como explicar

"O recurso nativo é a câmera, com a biblioteca oficial `expo-camera` . No `app.json` , o plugin configura a permissão do Android. Na tela, o `useCameraPermissions` pede a permissão em tempo de execução, e o `CameraView` lê só QR Codes. O QR contém o código da máquina, que é validado e abre a rota `/machine/código` . Uso um `useRef` como trava contra leituras repetidas, e a câmera só fica ligada com a aba em foco."

Android × Web: as 4 técnicas multiplataforma

A mesma base de código produz duas interfaces diferentes. Isso é feito com quatro técnicas, cada uma usada onde faz mais sentido.

Técnica 1 — Arquivos por plataforma (`.web.tsx` , `.android.tsx`)

O empacotador (Metro) escolhe o arquivo pela plataforma: no Android pega `arquivo.tsx` (ou `arquivo.android.tsx`); na Web, `arquivo.web.tsx`. Usado quando a diferença é **grande**, uma tela ou layout inteiro.

- `(app)/_layout.tsx` × `(app)/_layout.web.tsx` : Stack no Android × barra superior na Web.
- `(tabs)/_layout.tsx` × `(tabs)/_layout.web.tsx` : abas × sem abas.
- `scan.tsx` × `scan.web.tsx` : câmera × redirecionamento.

Regra do Expo Router: dentro de `src/app`, um `.web.tsx` só pode existir se houver também a versão sem extensão, para a rota existir em todas as plataformas.

Técnica 2 — `Platform.select` (o Alert, padrão do Pokémon)

`src/components/alert/index.ts`

```
import AlertIOS from "./index.ios";
import AlertAndroid from "./index.android";
import AlertWeb from "./index.web";

const AlertImplementation = Platform.select({
  ios: AlertIOS, android: AlertAndroid, web: AlertWeb, default: AlertWeb
});
export { AlertImplementation as Alert };
```

- Todas as telas usam `<Alert title message type visible onClose />`, com a mesma interface (`types.ts`).
- **Android** (`index.android.tsx`) chama o `Alert.alert` **nativo do sistema**, a caixa de diálogo padrão do Android.
- **Web** (`index.web.tsx`) desenha um `Modal` próprio com animação de fade e fechamento automático em 6 s, porque o navegador não tem um alerta nativo bonito.

Técnica 3 — `Platform.OS` (pequenas diferenças)

Um `if` dentro do mesmo arquivo, para diferenças pequenas:

- `authCookieIntegration.ts` : URL base (proxy na Web, API direta no Android).
- `home.tsx` : esconde o botão "Ler QR Code" na Web.
- `profile.tsx` : o botão "Sair" só aparece no Android (na Web ele fica na barra superior).
- Estilos: `const isWeb = Platform.OS === 'web'` ajusta tamanhos, `boxShadow` (Web) × `elevation` (Android).

Técnica 4 — `useWindowDimensions` (responsividade na Web)

Na Web, a janela pode ser larga (PC) ou estreita (celular abrindo o site). O hook devolve a largura atual, e o layout se adapta:

Onde	Largura	Comportamento
<code>auth-layout</code>	≥ 900 px	Painel preto com a logo à esquerda e o formulário à direita
<code>home.tsx</code> e <code>machine/[code].tsx</code>	≥ 820 px	Duas colunas (buscar minha reserva; detalhes ação)
<code>web-navbar</code>	< 640 px	Logo menor e nome do usuário escondido

Resumo visual das diferenças

Aspecto	ANDROID	WEB
Navegação	Abas inferiores + Stack com "voltar"	Barra superior + botão voltar do navegador
Encontrar máquina	QR Code ou digitar o código	Só digitar o código
Chamada da API de login	Direto para o onrender.com	Pelo proxy <code>/api/login/v1</code>
Alerta	Diálogo nativo do Android	Modal próprio
Layout	Tela cheia, uma coluna	Centralizado, duas colunas em telas largas
Onde fica "Sair"	Aba Perfil	Barra superior
Dados das máquinas	AsyncStorage (área do app)	AsyncStorage → localStorage

Fluxos completos, passo a passo

Juntando tudo: o que acontece em cada camada, do toque até o dado salvo.

Fluxo A — Cadastro (Web)

- 1 Tela** `register.tsx`: o usuário preenche o formulário; `validate()` confere e-mail, CEP (8 dígitos), senha ≥ 4 e confirmação.
- 2 Contexto**: `signup({ username, password, email, cep: só dígitos })`, e `isLoading = true` (o botão mostra carregando).
- 3 Integração**: `register()` → `authApi.post('/create')`. Na Web, a URL final é `localhost:8081/api/login/v1/create`.
- 4 Proxy**: `create+api.ts` → `forwardToAuthApi(request, '/create')` → API online.
- 5 API**: 201 + `Set-Cookie` + `{ userId, username, roles }`. O proxy devolve tudo, e o navegador guarda o cookie em `localhost`.
- 6 Integração**: `parseSession()` valida o corpo. **Contexto**: `setUser(response)` → `isAuthenticated = true`.
- 7 Rota**: o `(auth)/_layout` redesenha, vê que a pessoa está logada e faz `<Redirect href="/home">`. O usuário cai no Início já logado.

Fluxo B — Reservar pelo QR Code (Android)

- 1 Aba Escanear** (`scan.tsx`): `useCameraPermissions` → a permissão já concedida → mostra o `QrScanner` (câmera ativa porque a aba está em foco).
- 2 Câmera**: o `CameraView` detecta o QR e chama `onBarcodeScanned` com `data = "LVM-003"`; a trava bloqueia as leituras repetidas.
- 3 Tela**: `normalizeCode` + regex OK → `router.push('/machine/LVM-003')`. A câmera desliga, porque a aba perdeu o foco.
- 4 Tela da máquina**: `useLocalSearchParams()` lê `code = "LVM-003"` → `useMachine` → `getMachineByCode` → `loadMachines` (AsyncStorage) → a máquina livre.
- 5 Toque em "Utilizar máquina"**: `reserve()` → `runAction` → `reserveMachine(code, user)`.
- 6 Integração**: confere que está livre e que o usuário não tem outra reserva → grava `available:false` + `reservedBy { userId, username, reservedAt }` → `saveMachines` (AsyncStorage).
- 7 Tela**: a máquina atualizada volta para o estado, `isMine = true`, aparece "Cancelar reserva" e o `Alert` nativo "Máquina reservada!".
- 8 Início**: ao voltar, o `useFocusEffect` recarrega `getMyReservation`, e o card "Minha reserva" mostra a LVM-003.

Fluxo C — Outro usuário tenta usar a mesma máquina

- 1** O usuário A toca em "Sair": `signOut()` → `/logout` (com cookie) → `clearSession()` → o guard do `(app)` manda para `/`.

- 2 O usuário B faz login e abre a LVM-003 → `available: false` e `isMine: false` → a tela mostra "Em uso por A, desde 14:29", sem botão.
- 3 Se B tentasse reservar por outro caminho, `reserveMachine` lançaria `MachineError("já está em uso por A")`, porque a regra está na integração.

Fluxo D — Sessão expirada

- 1 O cookie tem `Max-Age=120`. Depois de 2 minutos, a API não o aceita mais.
- 2 Na próxima chamada protegida (hoje, o `/logout`), a API responde **403**.
- 3 O **interceptor** do `httpClientCookie` vê 403 numa rota que não é pública → `onUnauthorized()`.
- 4 O handler registrado pelo contexto executa `clearSession()` + `router.replace("/")`, e o usuário volta ao login.

Observação honesta

Como os dados das máquinas são locais, a única chamada protegida do app é o `/logout`. O interceptor já está pronto para quando existirem mais rotas protegidas (por exemplo, uma API real das máquinas); é a mesma estrutura do Pokémon.

Comparação com o projeto Pokémon

O professor avalia "a arquitetura vista em sala". Este capítulo mostra que ela foi seguida, e onde e por que houve adaptações.

O que foi mantido igual

Elemento	No Pokémon	No Lavamo
Pastas	<code>app</code> , <code>integration</code> , <code>context</code> , <code>hooks</code> , <code>components</code> , <code>constants</code> , <code>utils</code> , <code>@types</code>	as mesmas
Rotas protegidas	grupos <code>(auth)</code> e <code>(app)</code> , guard com <code>Redirect</code>	igual
Cliente HTTP com cookie	<code>httpClientCookie.ts</code> com <code>createApiCookie</code> e <code>setUnauthorizedHandlerCookie</code>	mesmo arquivo, mesmos nomes
Integração de autenticação	<code>authCookieIntegration.ts</code> com <code>register/login/logout</code>	mesmo arquivo, mesmos nomes
Contexto	<code>AuthCookieContext</code> com <code>useAuthCookie()</code> , <code>signIn/signUp/signOut</code>	igual
Alert por plataforma	<code>alert/index.ts</code> + <code>.android/.ios/.web</code> com <code>Platform.select</code>	copiado, restilizado em preto e branco
Componentes	pasta com <code>index.tsx</code> + <code>styles.ts</code> , estendendo props nativas	igual
Cores centralizadas	<code>constants/colors.ts</code>	igual
Alias de import	<code>@/*</code> → <code>src/*</code>	igual

O que foi adaptado, e por quê

Mudança	Motivo
Interceptor trata 401 e 403	A API do professor responde 403 com cookie inválido.
Exceção para <code>/auth</code> e <code>/create</code>	Senha errada também dá 403, e isso não é "sessão expirada".
<code>timeout: 60000</code>	O Render gratuito demora para acordar.
Rotas <code>/create</code> , <code>/auth</code> , <code>/logout</code>	São as rotas da API online do enunciado (o Pokémon usava a versão Docker local, com outros caminhos).
Cadastro sem <code>roles</code>	O <code>curl</code> do enunciado não tem esse campo.
Validação da resposta (<code>isAuthCookieResponse</code>)	Não confiar cegamente no formato da resposta.
Um único contexto de auth	O Pokémon mantinha dois (JWT + cookie). A atividade pede cookie, então ficou só o de cookie.
Proxy para a Web	O Pokémon usava a API em localhost (Docker), no mesmo site. Com a API online, o <code>SameSite=Lax</code> exige o proxy.
Sem o plugin de network security	O Pokémon liberava HTTP para um IP. A API online é HTTPS, então não é preciso.
Cadastro já entra no app	O <code>/create</code> da API online já devolve o usuário e o cookie.
Redirecionamento só nos guards	A tela de login do Pokémon navegava com <code>setTimeout</code> de 5 s; no Lavamo o guard reage à mudança do estado.

Decisões tomadas e seus motivos

A tabela para revisar antes da apresentação. Cada linha é uma pergunta "por que você fez assim?".

Decisão	Motivo	Alternativa descartada
Expo + Expo Router	Uma base para Android e Web; rotas por arquivo; API Routes para o proxy.	React Native CLI + React Navigation manual
API de login online	É a oferecida pelo professor e não exige instalar nada.	Docker local (mais peças na apresentação)
Proxy só na Web	O navegador descarta o cookie <code>SameSite=Lax</code> de outro site (testado). O Android não tem essa regra.	API direta (sem cookie na Web)
Cookie <code>HttpOnly</code> , token nunca no JS	Segurança contra XSS; é como a API do professor funciona.	Guardar o JWT no AsyncStorage (o JWTContext do Pokémon)
Sessão em memória	Sem rota "quem sou eu" e com o cookie de 2 minutos, lembrar o login mostraria uma sessão falsa.	Persistir o usuário no AsyncStorage
Cadastro entra direto	O <code>/create</code> já devolve o usuário e o cookie: menos passos.	Voltar para o login
<code>bd.json</code> + AsyncStorage	Simples, funciona offline, e as regras ficam isoladas na integração.	json-server (mais um servidor para rodar)
Limite de 1 reserva por usuário	Justiça numa lavanderia compartilhada.	Reservas ilimitadas
Câmera + QR como recurso nativo	Casa com o problema (QR colado na máquina); a <code>expo-camera</code> é oficial e roda no Expo Go.	GPS, Bluetooth
Web sem QR	Decisão de produto: no site o usuário digita o código.	Câmera também no navegador
Abas no Android, barra superior na Web	Cada plataforma com o padrão de navegação que o usuário já conhece.	Mesmo menu hambúrguer nas duas
Validação do código antes de navegar	Erro imediato para formato inválido, sem acessar os dados.	Deixar a tela da máquina acusar tudo
Regras de negócio na integração	Valem para qualquer tela e ficam num só lugar.	Regras dentro da tela
Preto e branco + logo trocável	Pedido do projeto; a logo é um arquivo, e as cores ficam em <code>colors.ts</code> .	—

Perguntas prováveis do professor

Organizadas pelos três temas que ele avisou: arquitetura, estrutura do código e decisões.

Arquitetura

P: Qual arquitetura você usou?

R: A arquitetura em camadas vista em aula: telas em `app`, estado global em `context`, lógica de tela em `hooks`, acesso a dados em `integration`, peças visuais em `components` e valores fixos em `constants`. Cada camada só chama a de baixo.

P: Para que serve a pasta `integration`?

R: É a única camada que sabe de onde vêm os dados. Ela esconde o axios, as URLs e o AsyncStorage atrás de funções como `login()` e `reserveMachine()`. Se a fonte mudar, só ela muda.

P: Qual a diferença entre `context` e `hooks`?

R: O context guarda um estado compartilhado por todo o app (o usuário logado). O hook encapsula uma lógica reutilizável com estado local de quem o usa (cada tela de máquina tem o seu `useMachine`).

P: Para que serve a pasta `utils`? Por que o proxy não está em `app`?

R: Funções auxiliares. O `authProxy.ts` é a função de repasse usada pelos três arquivos do proxy. Ele não fica em `app` porque tudo que está em `app` vira rota.

P: O que é um `_layout.tsx`?

R: A moldura das telas de uma pasta no Expo Router: define o navegador (Stack, Tabs ou Slot) e, no Lavamo, contém os guards que redirecionam conforme o login.

P: O que significam os parênteses em `(auth)` e `(app)`?

R: São grupos de rotas: organizam os arquivos sem aparecer na URL. `(auth)/register.tsx` é a URL `/register`.

P: Como o `app` impede acesso sem login?

R: O `(app)/_layout` lê `isAuthenticated` do contexto e, se for falso, retorna `<Redirect href="/">`. Isso vale para todas as telas dentro de `(app)`.

Cookie e autenticação

P: Como funciona o login com cookie?

R: O app manda usuário e senha para `/auth`. A API responde com os dados do usuário e um `Set-Cookie` com o JWT. Como o axios usa `withCredentials: true`, o navegador/app guarda o cookie e o envia sozinho nos próximos pedidos.

P: Onde fica guardado o token?

R: No cookie `access_token`, que é `HttpOnly`. O JavaScript não o lê; o app nunca toca no token. No contexto fica só `userId`, `username` e `roles`.

P: Para que serve o `withCredentials`?

R: Autoriza o envio e o recebimento de cookies em pedidos para outro endereço. Sem ele, o cookie de sessão não seria guardado nem enviado.

P: O que o interceptor faz?

R: Roda em toda resposta com erro. Se for 401 ou 403 numa rota protegida, chama o handler do contexto, que limpa a sessão e volta para o login. As rotas `/auth` e `/create` são ignoradas porque dão 403 em senha errada ou usuário duplicado.

P: Por que recarregar a página desloga?

R: A sessão fica em memória. A API não tem rota "quem sou eu", e o cookie `HttpOnly` não pode ser lido pelo JS; guardar o usuário localmente faria o app parecer logado com um cookie expirado (dura 2 minutos).

P: Por que o cadastro já entra no app?

R: Porque o `/create` da API devolve o usuário e o cookie, exatamente como o `/auth`. A pessoa já está autenticada.

Proxy

P: Por que você usou um proxy em vez de chamar a API online direto?

R: Porque o cookie vem com `SameSite=Lax`. Na Web, o site (`localhost`) e a API (`onrender.com`) são sites diferentes, e o navegador descarta o cookie. Eu testei: sem proxy, o `/logout` dava 403; com proxy, 200. O proxy faz o cookie vir do mesmo endereço do site.

P: O que são os arquivos auth+api, create+api e logout+api?

R: API Routes do Expo Router: código que roda no servidor, não no navegador. Cada um exporta uma função `POST` que chama `forwardToAuthApi` com o caminho correspondente (`/auth` , `/create` , `/logout`). O navegador chama `/api/login/v1/auth` , e o servidor repassa para a API online.

P: Mas o CORS da API não estava liberado?

R: Estava. CORS decide se o JavaScript pode ler a resposta, e por isso o login "funcionava". SameSite é outra regra, que decide se o cookie é guardado e enviado, e ela bloqueava o cookie.

P: Por que o Android não usa o proxy?

R: SameSite é uma regra de navegador. No Android, o módulo de rede nativo do React Native guarda e envia cookies sem essa restrição, então ele chama a API direto.

P: O que precisou configurar para o proxy funcionar?

R: `"web": { "output": "server" }` no `app.json` , que ativa as API Routes. No desenvolvimento, o próprio `npx expo start` roda o proxy.

P: Por que não usou o Docker?

R: Também resolveria, mas exigiria Docker rodando, o PC ligado, o celular na mesma rede, o IP no `.env` e HTTP liberado no Android. O proxy resolve com a API online, sem instalar nada.

Banco de dados

P: Onde ficam os dados das máquinas?

R: O `bd.json` é a carga inicial. Na primeira abertura, ela é copiada para o AsyncStorage (chave `@Lavamo:machines`), onde ficam as reservas. No Android fica na área do app; na Web, no localStorage.

P: Por que não grava direto no bd.json?

R: O arquivo vai empacotado dentro do app e é só leitura; o celular não acessa a pasta do PC, e o navegador não pode escrever arquivos.

P: Como o app sabe quem reservou?

R: Na reserva, grava em `reservedBy` o `userId` e o `username` que vieram do login da API. A tela compara esse `userId` com o do usuário logado (`isMine`).

P: Onde estão as regras (1 reserva, só o dono cancela)?

R: No `machineIntegration.ts`, em `reserveMachine` e `cancelReservation`. Quando uma regra é violada, elas lançam `MachineError`, e a mensagem aparece no Alert.

P: Qual a limitação desse banco?

R: Cada aparelho tem a própria cópia, então uma reserva feita no celular não aparece no navegador. Para compartilhar, bastaria trocar o interior do `machineIntegration.ts` por chamadas a uma API.

Câmera

P: Qual recurso nativo você usou, e com que biblioteca?

R: A câmera, com a `expo-camera`: o `CameraView` lê QR Codes e o `useCameraPermissions` pede a permissão do Android.

P: Como a permissão é configurada?

R: No `app.json`, o plugin `expo-camera` adiciona a permissão `CAMERA` ao manifesto do Android, define o texto do pedido (usado no iOS) e ativa o leitor (`barcodeScannerEnabled`). Em tempo de execução, o `useCameraPermissions` consulta e pede; se o usuário negou de vez (`canAskAgain` falso), o app explica como liberar.

P: O que o QR Code contém?

R: Só o código da máquina em texto, ex.: `LVM-002`. O app normaliza, valida com a regex `/^LVM-\d{3}$/` e abre `/machine/LVM-002`.

P: Por que tem um useRef no leitor?

R: A câmera detecta o mesmo QR várias vezes por segundo. O `useRef` é uma trava que ignora as leituras por 2 s, para não abrir a tela várias vezes.

P: Por que a câmera desliga ao trocar de aba?

R: `active={isFocused}`: o `CameraView` só é renderizado com a aba visível. Isso economiza bateria, respeita a privacidade e evita leituras em outra tela.

P: Como a Web fica sem QR?

R: `scan.web.tsx` redireciona `/scan` para `/home`, o botão "Ler QR Code" só aparece com `!isWeb`, e a barra da Web não tem Escanear.

Multiplataforma

P: Como o mesmo projeto gera duas interfaces diferentes?

R: Com quatro técnicas: arquivos por plataforma (`_layout.web.tsx`, `scan.web.tsx`), `Platform.select` (o Alert), `Platform.OS` para diferenças pequenas (a URL da API, os botões) e `useWindowDimensions` para a responsividade da Web.

P: Como o Alert funciona em cada plataforma?

R: O `alert/index.ts` escolhe com `Platform.select`: no Android usa o `Alert.alert` nativo; na Web, um Modal próprio. Todas as telas usam a mesma interface.

Glossário

Termo	Significado
API	Serviço que recebe pedidos HTTP e devolve dados (aqui, a API de login do professor).
API Route	Arquivo <code>+api.ts</code> do Expo Router que roda no servidor e responde a pedidos HTTP.
AsyncStorage	Armazenamento chave/valor persistente no aparelho (localStorage na Web).
axios	Biblioteca para fazer pedidos HTTP, com interceptors e configurações.
Context (React)	Mecanismo para compartilhar estado com todos os componentes sem passar props.
Cookie	Dado enviado pelo servidor (<code>Set-Cookie</code>) que o cliente guarda e reenvia (<code>Cookie</code>).
CORS	Regra do navegador que decide se um site pode ler a resposta de outro.
Expo Go	App que roda projetos Expo no celular sem compilar um APK.
Guard	Verificação num layout que redireciona quem não pode acessar a rota.
Hook	Função do React (<code>use...</code>) que encapsula estado e lógica reutilizável.
HttpOnly	Atributo do cookie que impede o JavaScript de lê-lo.
Interceptor	Função do axios que roda em todo pedido/resposta antes de chegar ao código.
JWT	Token assinado com os dados do usuário e a validade (fica dentro do cookie).
Metro	O empacotador do Expo: junta o código e escolhe os arquivos por plataforma.
Mock	Dado simulado, no lugar de um sistema real (o <code>bd.json</code>).
Proxy	Intermediário que recebe um pedido e o repassa para outro servidor.
Redirect	Componente do Expo Router que muda de rota ao ser renderizado.
SameSite	Atributo do cookie que limita o envio/armazenamento em pedidos entre sites diferentes.
Slot / Stack / Tabs	Formas de navegação: só a rota atual / telas empilhadas / barra de abas.
withCredentials	Opção do axios que permite enviar e receber cookies entre endereços.

Comandos úteis

Comando	O que faz
<code>npx expo start</code>	Sobe o servidor (app + proxy). <code>w</code> abre a Web; o QR do terminal abre no Expo Go.
<code>npx expo install <pacote></code>	Instala uma biblioteca na versão compatível com o SDK.
<code>npx tsc --noEmit</code>	Verifica os tipos do TypeScript.
<code>npx expo lint</code>	Verifica o padrão do código.
<code>npx expo-doctor</code>	Diagnostica dependências e configuração.

Roteiro de 1 minuto para abrir a apresentação

"O Lavamo resolve a falta de controle numa lavanderia de condomínio: cada máquina tem um QR Code. É um único projeto Expo que gera o app Android e o site. Segui a arquitetura em camadas vista em aula: telas, contexto, hooks e integração. O login usa a API do professor com cookie `HttpOnly` ; na Web, criei um proxy com API Routes, porque o navegador descartava o cookie `SameSite` vindo de outro domínio. O recurso nativo é a câmera, com a `expo-camera` , lendo o QR. E os dados das máquinas são mockados num `bd.json` , com as reservas salvas no `AsyncStorage`."